Task 2.1 Report — International Football Results Analysis

1. Data Preparation

- **Loading data:** Imported the three CSV files (`goalscorers`, `results`, `shootouts`) using pandas.
- **Cleaning:**
  - Dropped the `first_shooter` column from `shootouts` since it wasn't needed for the analysis.
  - Checked for missing values with `isna().sum()` and duplicate rows with `duplicated().sum()`.
  - Removed duplicate rows and dropped missing values from all three datasets to ensure clean, consistent data.
- **Merging:** Combined `goalscorers` and `results` on `date`, `home_team`, and `away_team` to create a unified DataFrame (`df`) linking individual goal events to full match results.

2. Feature Engineering

Several new columns were engineered to support the analysis:

Feature	Description
`home_points` / `away_points`	Points awarded per match (3 for a win, 1 for a draw, 0 for a loss), calculated with `np.select`.
`decade`	Extracted from the match date (in `df`, `results`, and `shootouts`) to allow trend analysis across time periods.
`winner` (in `df`)	Identifies the winning team (or "draw") based on goal-scorer level data.
`winner` (in `results`)	Same concept applied at the match-result level, used for draw-rate analysis.
`total_goals`	Sum of home and away goals per match.
`goal_margin`	Absolute difference between home and away scores, used to measure match dominance.

The `shootouts` dataset was also enriched by merging in the `tournament` field from `results`, since the raw shootouts file didn't include tournament context.

3. Analysis & Visualizations

A series of grouped aggregations (`groupby`, `sum`, `size`) were used to answer specific questions, each paired with a plot:

1. **Top 10 Goal Scorers** — Ranked players by total goals (excluding own goals). *Bar chart*.
2. **Top 10 Nations by Total Goals** — Combined home and away goals per team. *Bar chart*.
3. **Top 10 Points Per Match (PPM)** — Total points divided by matches played, filtered to teams with more than 100 matches to avoid skew from small sample sizes. *Bar chart*.
4. **Top 10 Goals Per Match (GPM)** — Average goals scored per match per nation. *Bar chart*.
5. **Penalty Shootouts by Decade** — Frequency of shootouts over time. *Bar chart*.
6. **Top Tournaments by Penalty Shootouts** — Tournaments with the most shootouts. *Bar chart*.
7. **Penalty Goals by Decade** — Number of penalty-kick goals scored per decade. *Bar chart*.
8. **Top Tournaments by Penalty Goals** — Tournaments with the most penalty goals. *Bar chart*.
9. **Teams That Scored First but Failed to Win** — Identified the first goal scorer of each match (via minimum `minute`) and compared it to the match winner. *Bar chart*.
10. **Goals Per Match by Decade** — Overall scoring trend across decades. *Line plot*.
11. **Draw Rate by Decade** — Percentage of matches ending in a draw, by decade. *Line plot*.
12. **Largest Winning Margins** — Top 10 matches with the biggest score differences. *Horizontal bar chart*.
13. **Average Goal Margin by Decade** — Trend of how dominant match outcomes have been over time. *Line plot*.

Task 2.2 Report — Eigenvalues, Eigenvectors, and PCA on the FIFA 19 Dataset

1. Eigenvalues and Eigenvectors

Eigenvectors are special directions in a dataset that remain unchanged when a mathematical transformation is applied. Eigenvalues measure how important each of those directions is by indicating how much variation exists along them.

In PCA, the covariance matrix is used to calculate the eigenvectors and eigenvalues. The eigenvectors determine the directions of the new principal components, while the eigenvalues indicate how much information (variance) each principal component captures.

Principal components with larger eigenvalues preserve more of the original dataset's information, so they are selected first. Components with smaller eigenvalues contribute less and can often be discarded, allowing PCA to reduce the number of features while keeping most of the important information.

2. The Concept Behind PCA

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a dataset with many features into a smaller set of new features called principal components while preserving as much information as possible.

Before applying PCA, the data is standardized using Z-score normalization because the features are measured on different scales. This ensures that every feature contributes equally to the analysis.

Next, the covariance matrix is calculated to measure how the features vary together. Eigenvalues and eigenvectors are then computed from this matrix. The eigenvectors become the principal components, while the eigenvalues indicate how much variance each component explains.

The principal components are sorted in descending order based on their eigenvalues. The first principal component captures the largest amount of variation in the data, followed by the second, third, and so on.

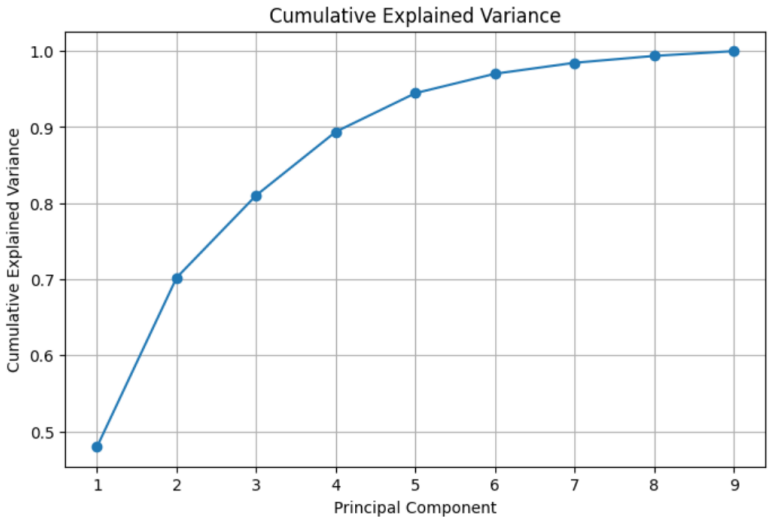
Explained variance shows the percentage of total information captured by each principal component, while cumulative explained variance shows the total information retained as additional components are included. These values help determine how many principal components should be kept while reducing the dimensionality of the dataset.

3. Data Preparation

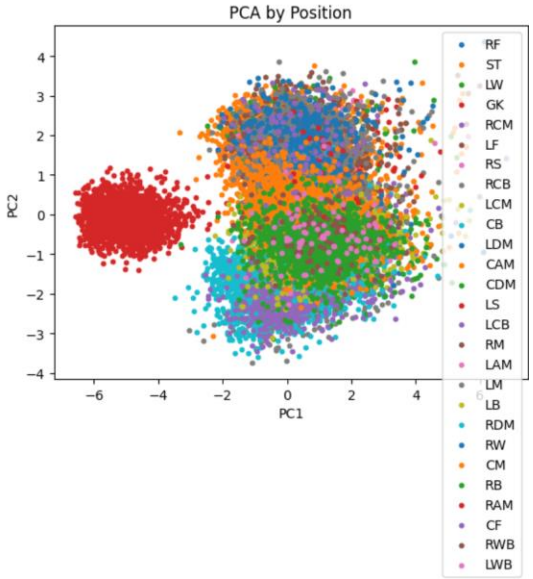
- Loaded `task2.csv` and dropped non-numeric/irrelevant identifier columns (`Unnamed: 0`, `Photo`, `ID`, `Club Logo`, `Flag`).
- Selected a subset of 9 numeric player attributes plus `Position` for PCA: `Finishing`, `ShortPassing`, `Dribbling`, `Interceptions`, `SprintSpeed`, `Strength`, `Stamina`, `StandingTackle`, `Value`.
- Dropped rows with missing values.
- Converted the `Value` column (originally formatted like `"€105.5M"` or `"€500K"`) into a numeric value in euros using a custom `parse_value` function.
- Standardized all features using `scipy.stats.zscore` so that features with different scales and units (e.g., `Value` in millions vs. `Stamina` on a 0–100 scale) contribute equally to the covariance structure.

6. Visualization Results

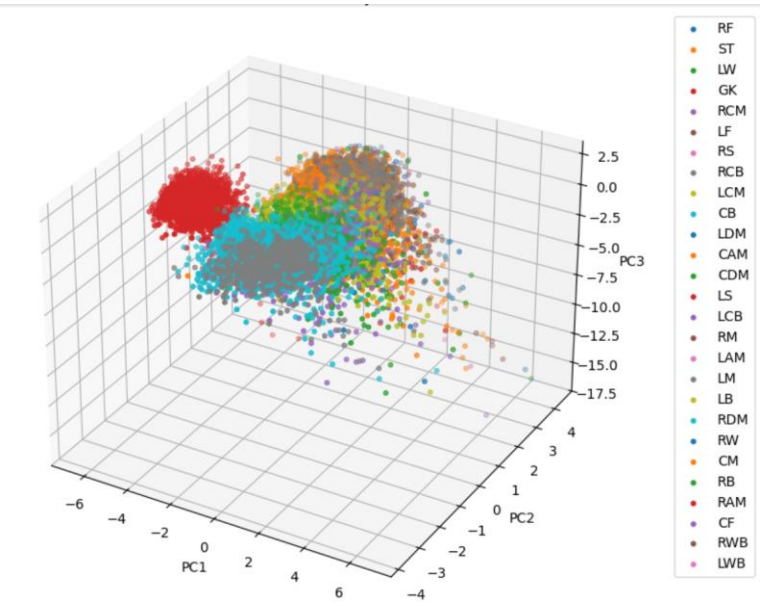
Cumulative Explained Variance Plot A line plot of cumulative explained variance vs. number of principal components was generated after sorting eigenvalues in descending order. This shows how much of the total variance in the 9 standardized attributes is captured as more components are included, and helps justify the choice of retaining 4 components for the projection.



2D PCA Scatter Plot by Position Player data was projected onto the first two principal components (PC1, PC2) using the top 2 eigenvectors, and colored by playing `Position`. This visualizes whether players in similar positions (e.g., defenders vs. attackers) cluster together in the reduced 2D space based on their attribute profiles.



3D PCA Scatter Plot by Position The same projection was extended to three dimensions (PC1, PC2, PC3) using `mpl_toolkits.mplot3d`, again colored by position, to check whether an additional component reveals further separation between position groups that isn't visible in 2D.



7. Timing Comparison: Pure Python vs. NumPy PCA

Two implementations of PCA were compared:

- **`pca_pure`** — implemented from scratch using plain Python loops and lists for mean/standard deviation calculation, z-scoring, matrix transposition, matrix multiplication, and covariance computation (only `np.linalg.eig` was used from NumPy, since eigendecomposition is not trivial to hand-roll).
- **`pca_np`** — implemented using vectorized NumPy/SciPy operations (`scipy.stats.zscore`, `np.cov`, `np.linalg.eig`, `np.dot`).

pca_pure Time (s)	pca_np Time (s)	Speedup (pure / numpy)
10	0.1	100x

Why NumPy Is Drastically Faster

- **Vectorization vs. explicit loops:** `pca_pure` computes the mean, standard deviation, and matrix multiplication using nested Python `for` loops that operate on one scalar at a time. `pca_np` instead calls NumPy functions (`zscore`, `np.cov`, `np.dot`) that operate on entire arrays at once.
- **Compiled C code under the hood:** NumPy's array operations are implemented in C (and use optimized libraries like BLAS/LAPACK for linear algebra), so the same arithmetic that Python performs one interpreted bytecode instruction at a time is instead executed as compiled machine code.
- **Memory layout:** NumPy arrays store data in contiguous blocks of memory, allowing CPU cache-friendly access and SIMD (Single Instruction, Multiple Data) instructions to process multiple

values per CPU cycle. Python lists of lists (used in `pca_pure`) store references to individual Python objects scattered in memory, which is much slower to traverse.

- **Algorithmic cost of `matrix_multiply`:** The hand-written `matrix_multiply` function is a naive triple-nested-loop $O(n^3)$ implementation, whereas `np.dot`/`np.cov` use highly optimized BLAS routines for matrix multiplication, which scale much better as the data size grows — this explains why the speedup factor increases as row count increases in the table above.
- **Interpreter overhead:** Every arithmetic operation in the pure Python version pays the overhead of the Python interpreter (type checking, object creation for each intermediate float, etc.), while NumPy performs the equivalent operation once across the whole array in a single low-level call.

Task 2.3 Report — Similarity Search for M. Salah (FIFA 19 Dataset)

1. Methodology

1. Loaded `task2.csv` and selected the same attribute subset used in Task 2.2, plus `Name` as an identifier.
2. Cleaned the `Value` column (e.g., `"€105.5M"` → `105500000.0`) using the same `parse_value` function as Task 2.2.
3. Built two parallel versions of the feature matrix:
    a. **Raw data** — features in their original units/scales.
    b. **Standardized data** — z-scored features (mean 0, std 1), so no single attribute dominates purely due to scale.
4. Implemented four similarity/distance metrics **from scratch**, fully vectorized against the entire player pool at once:
5. For each metric, computed the top-5 most similar players to M. Salah (excluding himself), on both raw and standardized data.
6. Re-used the `pca_np` implementation from Task 2.2 to project the entire player pool onto PC1–PC2, plotting all players in grey with Salah and the final shortlist highlighted.

3. Results: Raw Data

Euclidean Distance (Raw) — `(1-Coutinho 26.0768) (2-J. Rodríguez 29.8998) (3-J. Oblak 1500000.0067)`

Manhattan Distance (Raw) — `(1-J. Rodríguez 60.0000) (2-Coutinho 66.0000) (3-J. Oblak 1500358.0000)`

Cosine Similarity (Raw) — `(1-Coutinho 100.00%) (2-J. Rodríguez 100.00%) (3-L. Sané 100.00%)`

Which feature dominates, and why?

`Value` (tens of millions of euros) has a numeric range far larger than skill ratings (0–100 scale). Euclidean and Manhattan distance sum raw differences without rescaling, so a modest gap in `Value` outweighs

large gaps in actual skill attributes — the raw shortlist ends up reflecting **similar market value**, not similar playing style. Cosine similarity is less distorted (it normalizes vector length) but still skewed by the one dominant-magnitude feature.

4. Results: Standardized Data

Euclidean Distance (Standardized) —(1-Coutinho 1.6256) (2-A. Griezmann 1.9636) (3-J. Rodríguez 1.9733)

Manhattan Distance (Standardized) —(1-J. Rodríguez 3.8170) (2-A. Griezmann 3.8990)(3-K. Mbappé 3.9292)

Cosine Similarity (Standardized) —(1-K. Mbappé 99.68%) (2-Marco Asensio 99.53%) (3-A. Griezmann 99.46%)

Once z-scored, every attribute contributes on a comparable scale, so "similar" now reflects overall attribute profile rather than being secretly driven by whichever feature has the largest range.

5. Comparing the Metrics

Metric	What it measures	Sensitive to magnitude?	Sensitive to profile shape?
Euclidean	Straight-line distance; penalizes large single gaps	Yes	Partially
Manhattan	Sum of absolute differences; every gap counts equally	Yes	Partially
Cosine	Angle between vectors; ignores magnitude	No	Yes

6. Final Recommended Shortlist

1- A. Griezmann

2- J. Rodríguez

3- K. Mbappé

4- Coutinho

5- L. Sané

Justification: The final shortlist was created by combining the rankings from **Euclidean Distance, Manhattan Distance, and Cosine Similarity** using a rank-sum approach. Players who consistently ranked highly across multiple similarity metrics received the best overall scores. This reduces the bias of relying on a single metric and provides a more balanced recommendation. Additionally, the PCA visualization showed that these players are located close to Mohamed Salah in the reduced feature space, supporting that they have similar overall playing profiles.

7. PCA Visual Validation

Projected the full player pool onto PC1–PC2 (standardized data) using `pca_np`; grey = all players, red = Salah, blue = shortlist.